Software Architecture and Design Specification

Project: Online Bookstore

Version: 1.0

Authors: Vihaan Wadhwa, Nikhil Tej Vaddi, Abdullah A., Deepthi A E

Date: 29-09-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	29-09-2026	Vihaan Wadhwa, Nikhil Tej Vaddi, Abdullah A., Deepthi A E	First version of the architecture and design

Approvals

Role	Name	Signature / Date
Course Coordinator	Rajeshwari A.	

1. Introduction

1.1 Purpose

This document explains the architecture and design of our Online Bookstore project. It shows how the system is split into components, how those components talk to each other, and how the requirements from our SRS are covered by the design.

1.2 Scope

The design covers what buyers, sellers and admins can do on the site: register and log in, browse and search books, use the cart and checkout, see orders and tracking, manage seller listings, admin approval and inventory, and reviews, wishlist and email notifications. We do not design the inside of the payment gateway, email service or shipping partner. We only use their APIs.

1.3 Audience

Developers, testers and our course instructor.

1.4 Definitions

Term	Meaning
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix
ADR	Architecture Decision Record (a short note on a design choice and why we made it)
DRF, ORM	Django REST Framework, Object-Relational Mapping (Django ORM)
JWT	JSON Web Token, used to keep a user logged in

Term	Meaning
RBAC	Role-Based Access Control (Buyer, Seller, Admin)
PCI-DSS	Payment card security standard. We stay out of its scope by never storing card data
TLS	Encryption used for HTTPS (version 1.2 or higher)
STRIDE	A threat checklist: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
WCAG	Web accessibility guidelines (we target 2.1 AA)
RPO / RTO	Recovery Point Objective / Recovery Time Objective (how much data we can lose / how long recovery can take)

2. Document Overview

2.1 How to use this document

Section 3 covers the architecture and section 4 covers the detailed design (sequence diagrams, APIs, error handling and UX). We use the same requirement IDs (BOOK-F, BOOK-NF, BOOK-SR) and test case IDs as the SRS, so the documents can be matched easily.

2.2 Related Documents

- Online Bookstore SRS v1.0 (the RTM is in its section 8)
- Software Test Plan (Lab 3)

3. Architecture

3.1 Goals & Constraints

Goals:

- Keep user and payment data safe: HTTPS with TLS 1.2+, hashed passwords, no card data stored, and role checks on the server (BOOK-SR-001 to 005).
- Keep pages fast: catalogue and search under 3 seconds and checkout under 5 seconds for 90% of requests (BOOK-NF-001).
- Support 500 users at the same time and 99.5% uptime per month (BOOK-NF-002, NF-003).
- Take daily backups with RPO of 24 hours and RTO of 4 hours (BOOK-NF-006).
- Keep the code easy to test (70% coverage, BOOK-NF-005) and the UI usable on mobile and desktop (WCAG 2.1 AA, BOOK-NF-004).

Constraints:

- Must use Python/Django with PostgreSQL.
- Payments use a sandbox gateway (Razorpay or Stripe), so no real transactions, and card details must never be stored on our server.
- It is an academic project, so the scope is limited to the features in the SRS.

3.2 Stakeholders & Concerns

Stakeholder	Main concerns
Buyers	Safe payment, fast browsing and checkout, correct order status
Sellers	Easy listing; nobody else can edit their listings
Admins	Approve listings and manage users and inventory

Stakeholder	Main concerns
Developers	Modules that are easy to change and test
Testers / instructor	Design that matches the SRS and test plan

3.3 Component (UML) Diagram

Figure 1 shows the components and the external services they use. The names WebUI, APIGateway, AuthModule, CartModule, CheckoutModule and SellerModule are the same as in the SRS traceability matrix. We added the other modules that the remaining requirements need.

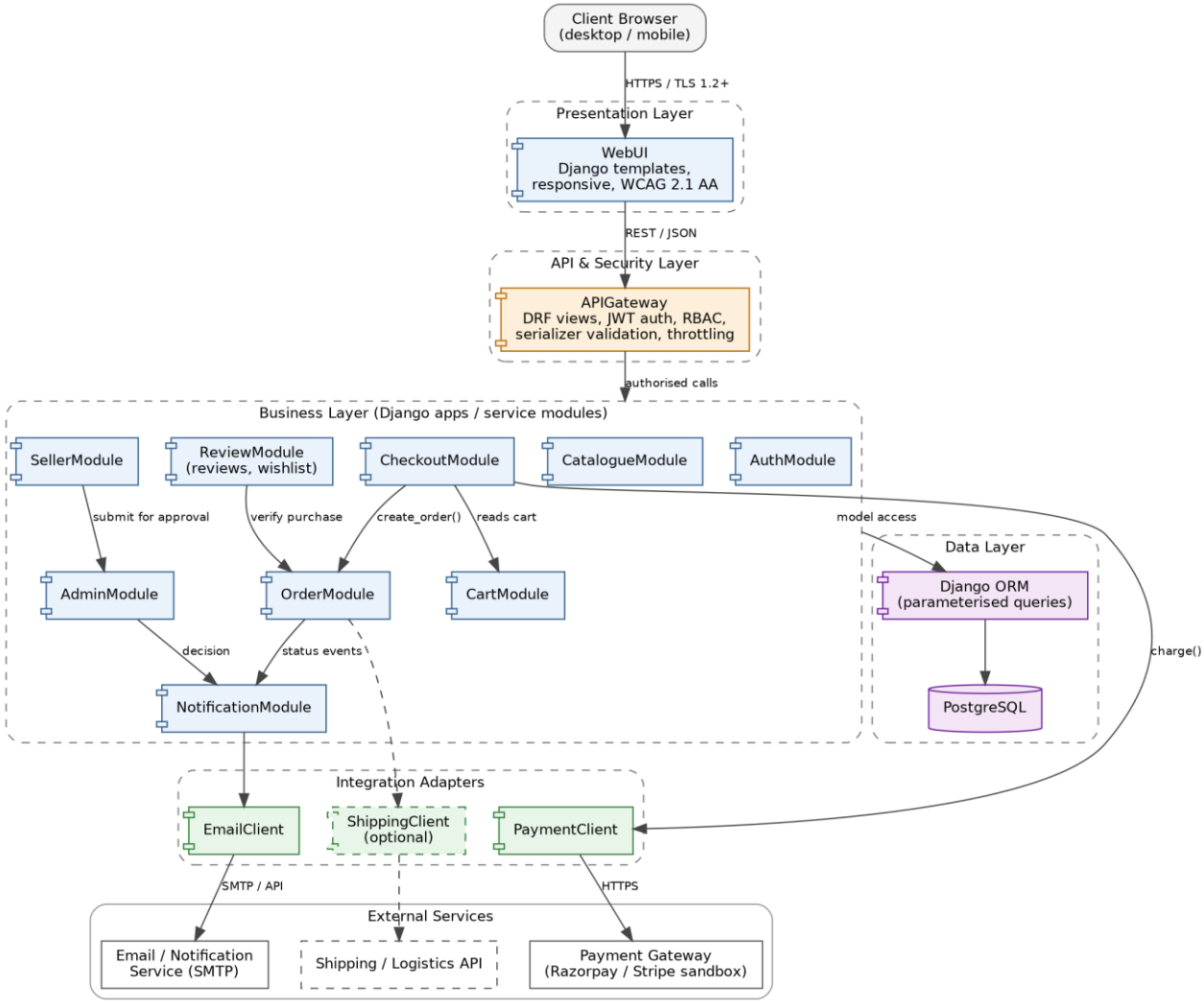


Figure 1: Component diagram

Figure 2 shows the main data the modules share through the Django ORM. The Payment table only keeps the gateway reference, never card details (BOOK-SR-003).

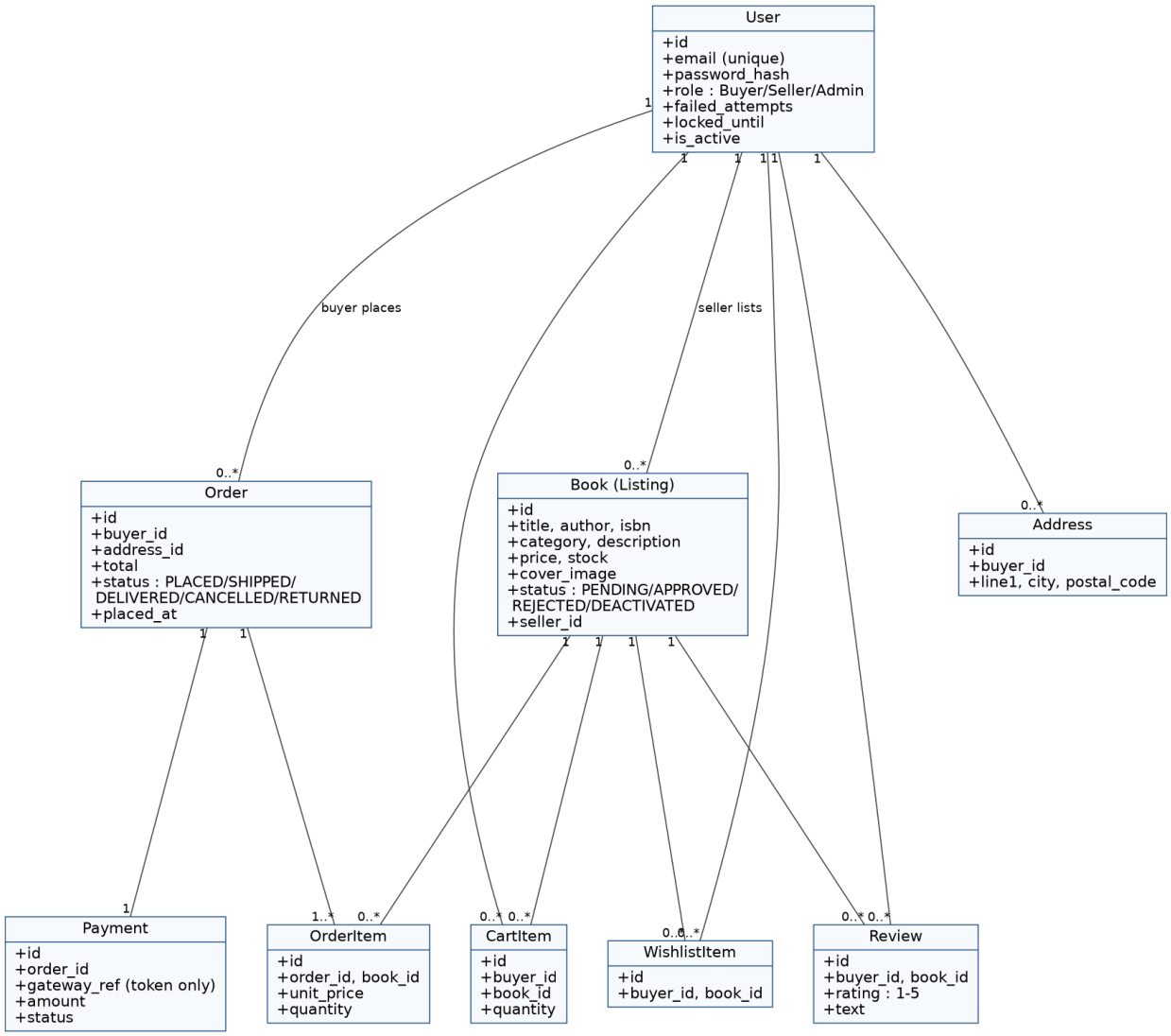


Figure 2: Class diagram of the main data

3.4 Component Descriptions

Component	What it does	Requirements
WebUI	Django templates for the pages (catalogue, cart, checkout, orders, account). Responsive and keyboard friendly.	F-005 to F-013, NF-004
APIGateway	Entry point for the REST API. Checks the JWT, checks the user role, validates input and returns errors in one standard format.	SR-004, SR-005
AuthModule	Register, login, JWT, lockout after 5 failed attempts, password reset by email, password hashing.	F-001 to F-004, SR-002
CatalogueModule	Catalogue with pagination, search and filters, book detail page. Only approved listings are shown.	F-005 to F-007
CartModule	Add, change quantity and remove items. Totals are calculated on the server.	F-008
CheckoutModule	Checks address, payment method and stock, then charges through the gateway. If payment fails the cart is not changed.	F-009, F-010, SR-003
OrderModule	Creates the order, order history, status changes, tracking, cancel/return within the allowed days.	F-011 to F-013
SellerModule	Seller can create, update and remove only their own listings. New or edited listings wait for approval.	F-014, F-015

Component	What it does	Requirements
AdminModule	Approve or reject listings, adjust inventory, deactivate listings and accounts.	F-016, F-017
ReviewModule	Ratings and reviews (only for buyers who bought the book) and the wishlist.	F-018, F-019
NotificationModule	Sends emails for order updates and listing decisions.	F-020
PaymentClient, EmailClient, ShippingClient	Small wrappers that call the payment, email and (optional) shipping APIs.	F-010, F-012, F-020
Django ORM + PostgreSQL	Stores all the data. Only ORM queries are used, no raw SQL.	SR-005, NF-006

3.5 Chosen Architecture Pattern and Rationale

We chose a layered architecture with four layers: presentation (WebUI), API (APIGateway), business logic (the modules) and data (ORM and PostgreSQL). Everything runs as one Django project, with one Django app for each module.

We picked this because a single project is easier for a small team to build, test and deploy, and checkout can be done in one database transaction. We rejected microservices because they need separate services and network calls between them, which is too much for a project with about 500 users.

ADR	Decision	Reason
ADR-001	Layered design in one Django project	Simple to build and test; checkout stays in one transaction
ADR-002	JWT for login on top of Django users	No session storage needed for the API
ADR-003	Use gateway tokens, never store card data	Keeps us out of PCI-DSS scope

3.6 Technology Stack & Data Stores

Part	Technology
Backend	Python, Django, Django REST Framework
Frontend	Django templates (responsive HTML/CSS)
Login and security	django.contrib.auth (PBKDF2/Argon2 hashing), JWT, HTTPS
Database	PostgreSQL through the Django ORM
External services	Razorpay or Stripe sandbox, SMTP email service, optional shipping API
Testing	Django TestCase / pytest-django with a coverage report
Hosting	Render, Heroku or AWS with HTTPS and daily backups

3.7 Risks & Mitigations

Risk	Mitigation
Payment gateway is slow or down during checkout	Use timeouts and show an error. The cart stays unchanged so the buyer can try again (BOOK-F-010).
Two buyers buy the last copy at the same time	Check stock again and lock the row inside the order transaction. If stock ran out after payment, refund and show an error.
Brute-force login attempts	Lock the account for 15 minutes after 5 failed attempts (BOOK-F-003).

Risk	Mitigation
Email service fails	Send emails after the order is saved and retry later. A failed email should never cancel an order.
Data loss	Daily backups and a restore test (BOOK-NF-006).

3.8 Traceability to Requirements

Req ID	Requirement	Component(s)	Test case
BOOK-F-001	Register account	AuthModule	TC-Auth-01
BOOK-F-002	Login and token	AuthModule, APIGateway	TC-Auth-02
BOOK-F-003	Lock account after 5 failures	AuthModule	TC-Auth-03
BOOK-F-004	Password reset link	AuthModule, NotificationModule	TC-Auth-04
BOOK-F-005	Paginated catalogue	CatalogueModule, WebUI	TC-Cat-01
BOOK-F-006	Search and filters	CatalogueModule	TC-Cat-02
BOOK-F-007	Book detail page	CatalogueModule, ReviewModule	TC-Cat-03
BOOK-F-008	Cart add/update/remove	CartModule	TC-Cart-01
BOOK-F-009	Address and payment method	CheckoutModule	TC-Cart-02
BOOK-F-010	Payment and order confirmation	CheckoutModule, PaymentClient, OrderModule	TC-Cart-03
BOOK-F-011	Order history	OrderModule	TC-Ord-01
BOOK-F-012	Shipment tracking	OrderModule, ShippingClient	TC-Ord-02
BOOK-F-013	Cancel / return window	OrderModule	TC-Ord-03
BOOK-F-014	Seller creates listing	SellerModule	TC-Sell-01
BOOK-F-015	Seller edits/removes own listing	SellerModule, APIGateway	TC-Sell-02
BOOK-F-016	Admin approves/rejects listing	AdminModule, NotificationModule	TC-Admin-01
BOOK-F-017	Admin inventory / deactivate	AdminModule	TC-Admin-02
BOOK-F-018	Rating and review	ReviewModule, OrderModule	TC-Rev-01
BOOK-F-019	Wishlist	ReviewModule	TC-Rev-02
BOOK-F-020	Status emails	NotificationModule, EmailClient	TC-Rev-03
BOOK-NF-001	Response time targets	WebUI, APIGateway	TC-Perf-01
BOOK-NF-002	500 concurrent users	Deployment, database	TC-Perf-02
BOOK-NF-003	99.5% availability	Deployment, monitoring	Ops report
BOOK-NF-004	Responsive, WCAG 2.1 AA	WebUI	TC-UX-01
BOOK-NF-005	70% test coverage	All modules	CI report
BOOK-NF-006	Daily backup, RPO/RTO	PostgreSQL, ops procedure	TC-Ops-02
BOOK-SR-001	TLS 1.2+	Deployment, PaymentClient, EmailClient	TC-Sec-01
BOOK-SR-002	Password hashing	AuthModule	TC-Sec-02
BOOK-SR-003	No card data stored	CheckoutModule, PaymentClient	TC-Sec-03
BOOK-SR-004	Role-based access control	AuthModule, APIGateway	TC-Sec-04

Req ID	Requirement	Component(s)	Test case
BOOK-SR-005	Injection/XSS protection	APIGateway, Django ORM	TC-Sec-05

3.9 Security Architecture

We used the STRIDE checklist to find the main threats and matched each one with a control.

Threat	Example	How we handle it
Spoofing	Someone logs in as another user	Hashed passwords, signed JWTs, lockout after 5 failures
Tampering	Changing the price or total in a request	Totals are calculated on the server; HTTPS
Repudiation	A user denies placing an order or an admin denies a decision	Orders, payments and admin actions are logged with user and time
Information disclosure	Leak of passwords or card data	Card data is never stored, passwords are hashed, no secrets in logs
Denial of service	Flooding the login or checkout	Lockout, request limits, pagination
Elevation of privilege	A buyer calls a seller or admin API	Role checks on the server for every endpoint

SQL injection and XSS are handled by the Django ORM, template auto-escaping and DRF serializer validation (BOOK-SR-005). Keys and passwords (gateway keys, email login, Django secret key) go in environment variables, not in the repo.

4. Design

4.1 Design Overview

Every request goes through the same path: browser, WebUI, APIGateway (login check, role check, input validation), the module, the ORM and then the database. Emails are sent only after the database changes are saved. Listings and orders follow a fixed set of statuses:

Item	Statuses
Book listing	PENDING, then APPROVED or REJECTED by an admin. If the seller edits an approved listing it goes back to PENDING. An admin can DEACTIVATE a listing. Only APPROVED listings show in the catalogue.
Order	PLACED, SHIPPED, DELIVERED. A buyer can CANCEL or RETURN only inside the allowed days (for example 7 days).

4.2 UML Sequence Diagrams

We show three flows that are specific to our project.

Flow 1: Checkout and payment (BOOK-F-008 to F-010, F-020)

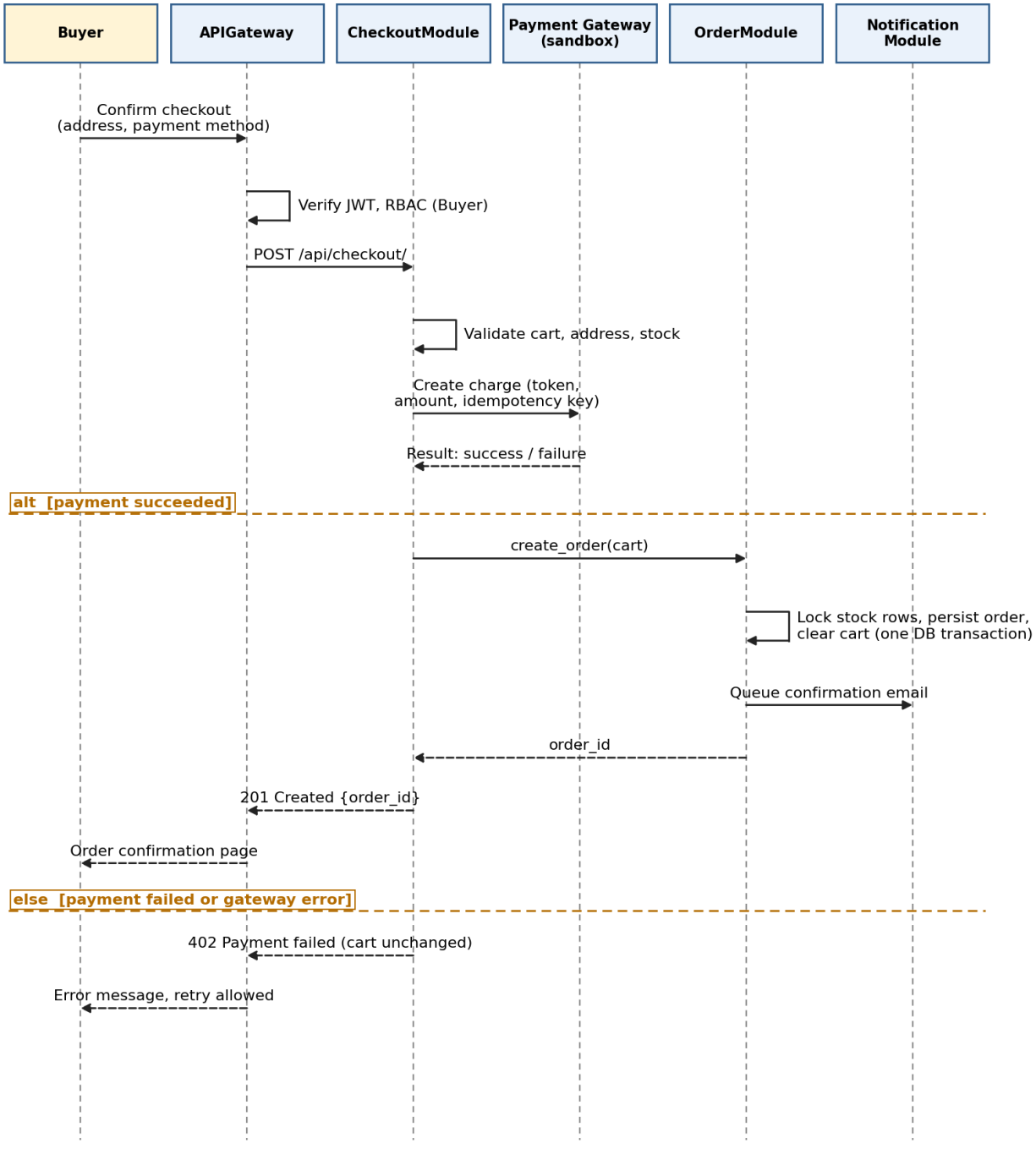


Figure 3: Checkout and payment

Flow 2: Seller listing and admin approval (BOOK-F-014, F-016)

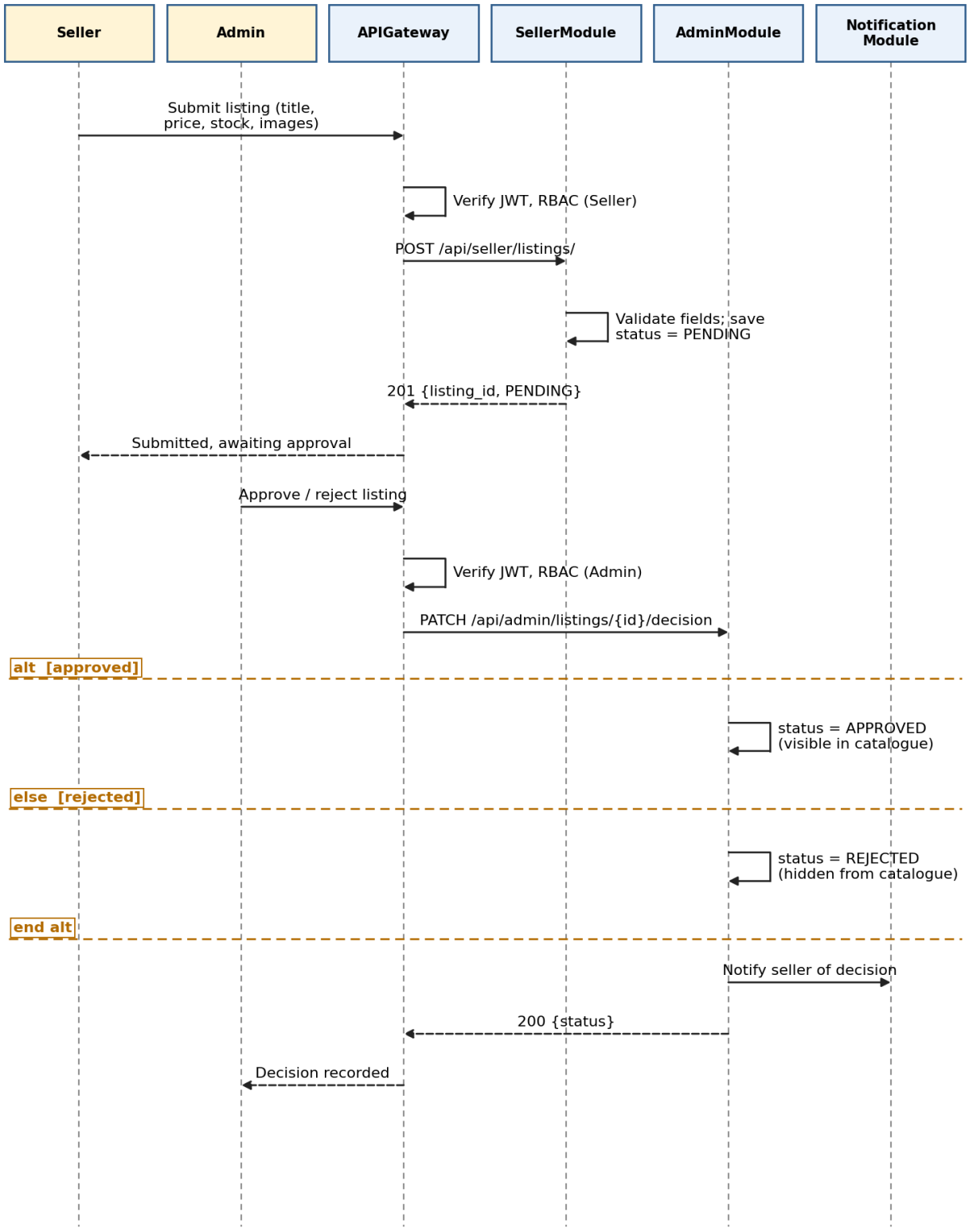


Figure 4: Listing submission and approval

Flow 3: Login with account lockout (BOOK-F-002, F-003)

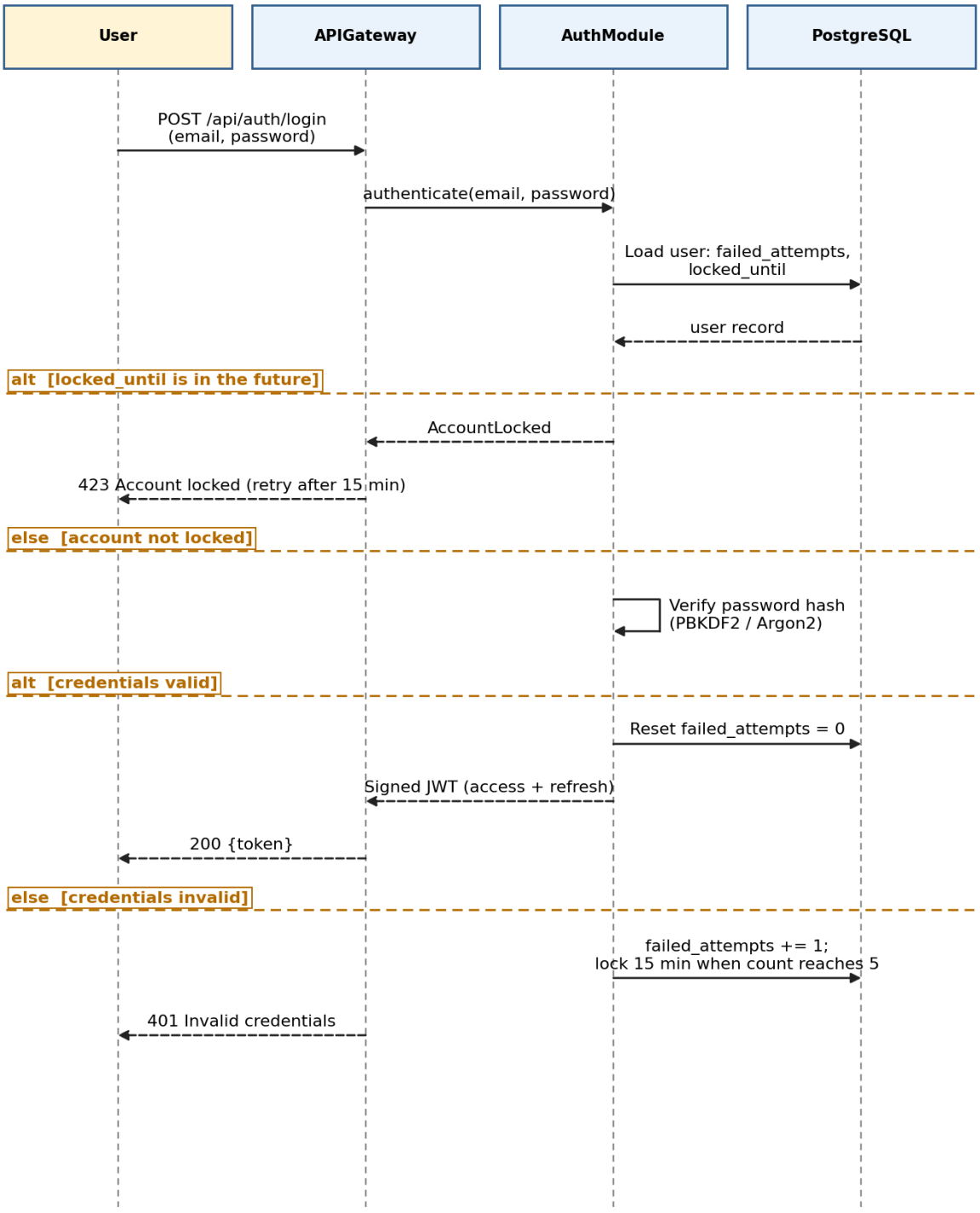


Figure 5: Login and lockout

4.3 API Design

All endpoints use HTTPS and JSON. Protected endpoints need the header Authorization: Bearer <access_token>. Below are the interfaces for AuthModule and CheckoutModule, followed by a short list of the other endpoints.

AuthModule

Endpoint	Method	Access	Request	Response	Errors
/api/auth/register/	POST	Public	{email, password, role}	201 {user_id, role}	400 invalid data; 409 email already used
/api/auth/login/	POST	Public	{email, password}	200 {access, refresh, role}	401 wrong credentials; 423 account locked
/api/auth/password-reset/	POST	Public	{email}	202	429 too many requests
/api/auth/password-reset/confirm/	POST	Reset link	{token, new_password}	200	400 link expired (30 min) or already used

CheckoutModule

Endpoint	Method	Access	Request	Response	Errors
/api/checkout/	POST	Buyer	{address_id, payment_method_token}	201 {order_id, total, status}	400 missing address or payment method; 402 payment failed (cart unchanged); 409 out of stock; 503 gateway not available

Example for POST /api/checkout/:

```
Request:
{ "address_id": 12, "payment_method_token": "tok_sandbox_abc123" }

Success (201):
{ "order_id": 4021, "total": "1298.00", "status": "PLACED" }

Payment failed (402):
{ "error": { "code": "PAYMENT_FAILED", "message": "Payment was declined. Your cart is unchanged."
} }
```

Other endpoints

Endpoint	Method	Who	Purpose	Req
/api/books/	GET	Public	Catalogue, search and filters	F-005, F-006
/api/books/{id}/	GET	Public	Book detail	F-007
/api/cart/items/	POST, PATCH, DELETE	Buyer	Add, update, remove cart items	F-008
/api/orders/	GET, POST	Buyer	Order history, cancel or return	F-011, F-013
/api/seller/listings/	POST, PATCH, DELETE	Seller	Manage own listings	F-014, F-015
/api/admin/listings/{id}/decision/	PATCH	Admin	Approve or reject a listing	F-016
/api/books/{id}/reviews/	POST	Buyer	Add a review	F-018

4.4 Error Handling, Logging & Monitoring

- Errors always come back in the same format: { "error": { "code", "message" } } with the right HTTP status. Messages do not show internal details.

- Calls to the payment, email and shipping services use timeouts. Emails are retried a few times. A payment is never retried blindly, to avoid charging twice.
- Logs keep the request, user id, status and time. Passwords, tokens and card data are never logged (BOOK-SR-002, SR-003).
- We monitor uptime, response time, checkout failures and failed logins.

4.5 UX Design

- Responsive pages for mobile and desktop, forms that work with the keyboard, labels and good contrast (WCAG 2.1 AA).
- Main menu: Catalogue, Cart, My Orders, Wishlist, Account. Sellers and admins get their own dashboard.
- Checkout goes step by step (cart, address, payment, confirmation) and shows clear error messages.

4.6 Open Issues & Next Steps

- Decide if the shipping API will be implemented or if tracking status will be updated manually.
- Decide how to send emails in the background (Celery or a simple task queue).
- Future ideas: two-factor login, wishlist price alerts, better search, caching.

5. Appendices

5.1 Glossary

See section 1.4.

5.2 References

- IEEE 42010, OWASP Top 10, NIST SP 800-160
- Online Bookstore SRS v1.0, Django and DRF documentation

5.3 Tools

Graphviz (component and class diagrams), Python/Matplotlib (sequence diagrams).